Git_GitHub

Git & GitHub

목 차

- 버전 관리의 필요성
- Git 기본 개념
- Git 설치와 초기 설정
- 로컬에서 Git 사용하기
- 브랜치 이해하기
- GitHub 기초
- 협업 실습
- 자주 사용하는 명령어

과정 목표

: Git과 GitHub를 활용해 코드를 안전하게 관리하고 협업할 수 있는 기본 역량을 갖춘다

버전 관리(Version Control)란?

- 파일이나 코드의 변경 이력을 체계적으로 기록하고 관리하는 것
- 언제, 누가, 무엇을 바꿨는지 알 수 있고, 필요하면 과거 상태로 되돌릴 수 있음

버전 관리가 필요한 이유

- 변경 이력을 추적하고, 실수를 복구하며, 여러 사람이 충돌 없이 협업하기 위해

버전 관리 활용

- 기능을 추가했다가 버그가 생겼을 때 이전 정상 동작 버전으로 바로 되돌릴 수 있다
- 팀원이 동시에 같은 파일을 수정해도 누가 무엇을 바꿨는지 확인하고 충돌을 해결할 수 있다
- "최종_진짜최종_v3" 같은 파일을 만들지 않고도 변경 과정을 깔끔하게 관리할 수

Git vs. 파일 복사

파일 복사 방식

- final.py, final_v2.py, 진짜최종.py처럼 파일이 늘어남
- 변경 이유와 내용이 남지 않음
- 협업 시 어떤 파일이 최신인지 알기 어려움

Git 사용

- 변경 내용이 커밋 단위로 기록됨
- 언제든 특정 시점으로 되돌릴 수 있음
- 여러 사람이 동시에 작업 가능
- 변경 이력과 책임이 명확함

Git vs. GitHub

Git
- 버전 관리를 위한 도구(프로그램)
- 내 컴퓨터에서 동작

GitHub
- Git 저장소를 올려두는 웹 서비스
- 코드 공유 및 협업 플랫폼
- Pull Request, Issue, Review 기능 제공

Git은 버전 관리 엔진,GitHub는 협업과 공유를 위한 서비스

Repository(레포지토리)

- **파일과 그 변경 이력을 함께 관리하는 저장소**
- **프로젝트 전체와 모든 버전 기록이 들어 있음**

- 로컬 저장소: 내 컴퓨터에 있는 저장소
  - 프로젝트 폴더로 이동
  - **git init** 실행 → 해당 폴더가 Git 레포지토리가 됩니다
- 원격 저장소: GitHub 같은 서버에 있는 저장소
  - GitHub에서 새 Repository 생성
  - **git clone**으로 내 컴퓨터에 복사

Commit(커밋)

- **변경 사항을 하나의 기록으로 저장하는 단위(= 하나의 버전)**

- "무엇을 왜 바꿨는지"를 메시지로 남김
- 특정 시점의 프로젝트 상태를 저장
- 되돌아갈 수 있는 기준점 역할

- 커밋 생성: **git commit -m "회원가입 기능 추가"**

Branch(브랜치)

- **독립적으로 작업할 수 있는 작업 공간**

- 기존 코드를 건드리지 않고 기능 개발 가능
- 여러 기능을 동시에 개발할 수 있음
- 작업이 끝나면 병합(merge)함

- 브랜치 생성: **git branch feature/login**
- 브랜치 전환: **git switch feature/login**

HEAD(헤드)

- **현재 내가 작업 중인 위치를 가리키는 포인터**

- 보통 현재 브랜치의 최신 커밋을 가리킴
- HEAD가 움직인다는 것은 작업 기준이 바뀐다는 의미



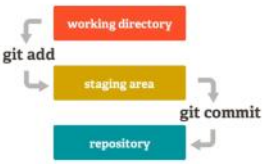
프로그램 다운 받음.
맥에서 할려면:

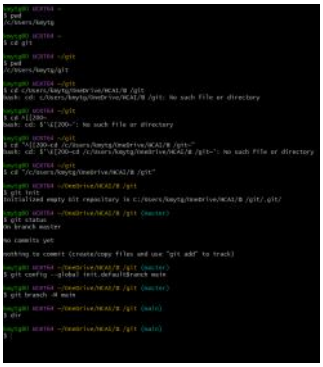
https://brew.sh/
Powershell.

OZ. AI 웹스케어 초격차 캠프 Git 기본 개념

Working Directory

- 실제로 파일을 수정하고 작업하는 공간
- 아직 Git에 기록되지 않은 변경 사항이 있는 상태



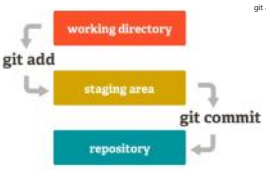


OZ. AI 웹스케어 초격차 캠프 Git 기본 개념

Staging Area

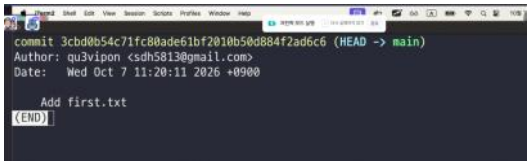
- 다음 커밋에 포함할 변경 사항을 미리 모아두는 공간

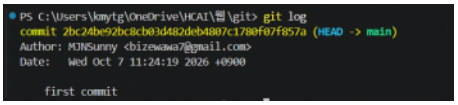
Working directory 에서 staging area 가는법:
git add *file name*

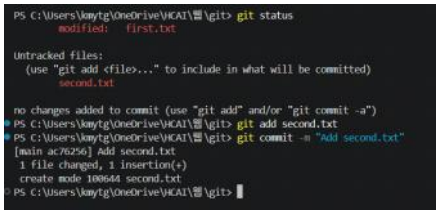


Vs code, shell....(git bash에서 코드 불러오기 커맨드, ctrl shift p) 못함
Autosave 못함

git log









OZ. AI 웹스케어 초격차 캠프 Git 설치와 초기 설정

Git 설치

- macOS: brew install git
- Windows: Git for Windows 설치
  - Git for Windows/x64 Setup.
- 설치 확인: git --version

OZ. AI 웹스케어 초격차 캠프 Git 설치와 초기 설정

GitHub 회원가입

- https://github.com/ 접속해서 회원가입
  - name, email 기록

난 왜 달아버린거 같지.. 강사님꺼는 날아있었음.



<- 아 ! 아니 밑에 있었음. 아래쪽 화살표 쓰면 됨.

OZ. AI 웹스케어 초격차 캠프 Git 설치와 초기 설정

git config

Git의 기본 동작을 설정하는 명령어

전역 설정: 모든 저장소에 적용
- git config --global user.name "홍길동"
- git config --global user.email "hong@example.com"

로컬 설정: 특정 저장소에만 적용
- 프로젝트마다 다른 계정이나 역할을 써야 할 때 사용(회사 프로젝트와 개인 프로젝트 구분)
- git config user.name "홍길동"
- git config user.email "hong@example.com"

git init

현재 폴더를 Git 레포지토리로 초기화

git init
- .git 폴더가 생성됨
- 현재 폴더를 기준으로 버전 관리 시작

git status

현재 레포지토리의 상태를 확인

git status
- 수정된 파일
- 스테이징 여부
- 커밋 가능한 변경 사항 확인

git add

변경된 파일을 Staging Area로 올리는 명령어

git add 파일명 / git add .
- 커밋에 포함할 변경 사항을 선택하는 단계
- 아직 커밋이 생성된 것은 아님

git commit

Staging Area에 있는 변경 사항을 하나의 버전으로 기록

git commit -m "커밋 메시지"
- 커밋은 되돌릴 수 있는 기준점 역할
- 예: 게임 save file

커밋 메시지 작성 규칙

좋은 커밋 메시지는 무엇을 했는지 한눈에 알 수 있어야 한다

권장 규칙
- 명확하고 간결하게 작성
- 동사로 시작
- 한 커밋 = 한 작업

예시
- Add login API
- Fix password validation bug
- Refactor user service

.gitignore

.gitignore는 버전 관리에서 제외할 파일을 지정하는 설정 파일

불필요한 파일이 커밋되는 것을 방지
- 환경 변수 파일, 빌드 결과물, 개인 설정 파일 등은 보통 제외

예시
- .env
- __pycache__/
- *.log

<주의> 이미 커밋된 파일은 Git이 계속 추적하기 때문에, git rm --cached 파일명

(퀴즈) Git에서 하나의 작업 단위를 기록한 것은 무엇인가요?

A. branch
B. status
C. commit
D. version

(퀴즈) Git을 사용하는 가장 큰 목적은 무엇인가요?

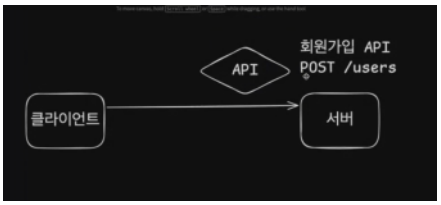
A. 파일을 압축하기 위해
B. 코드를 서버에 배포하기 위해
C. 코드를 다른 사람과 공유하기 위해
D. 코드 변경 이력을 관리하기 위해

(퀴즈) Working Directory와 Staging Area의 차이를 가장 잘 설명한 것은?

A. 스테이징은 임시 백업이다
B. 스테이징은 커밋할 파일을 선별하는 공간이다
C. 둘 다 원격 저장소이다
D. 스테이징은 자동으로 변경 사항을 커밋해주는 공간이다

임시백업은 예매,
원격이 아닌 로컬
자동이 아닌 수동

API 예기 1시

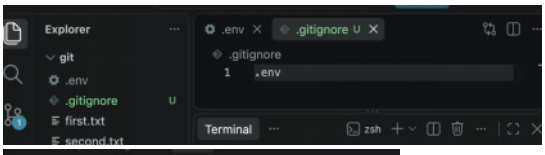


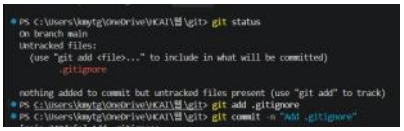


Branch(브랜치)

- 기존 작업과 분리된 독립적인 작업 흐름
- 기능 개발, 실험, 수정 작업을 안전하게 진행할 수 있음
- 협업과 안정적인 배포를 위해서는 브랜치를 잘 관리하는 것이 중요

주요 명령어
- 브랜치 목록 확인: git branch
- 브랜치 생성: git branch feature/ui
- 브랜치 전환: git switch feature/ui





브랜치 관리 전략

어떻게 브랜치를 나누고 합칠지에 대한 약속

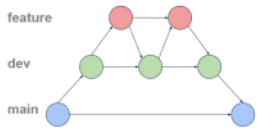
예시



어떻게 브랜치를 나누고 합칠지에 대한 약속

예시

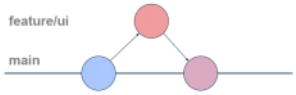
- main: 배포 가능한 안정 버전
- dev: 개발 중인 기능을 테스트하는 버전
- feature/*: 기능 개발
- hotfix/*: 긴급 수정



브랜치 병합

git merge
- 다른 브랜치의 작업 내용을 현재 브랜치로 합치는 명령어

git merge feature/ui



Fast-forward vs. 3-way merge

Fast-forward merge
- 브랜치가 직선으로 이어질 때
- 단순히 커밋 포인터만 이동

3-way merge
- 브랜치가 갈라졌다가 다시 합쳐질 때
- 병합 커밋이 생성됨

분기가 없으면 **Fast-forward**, 분기가 있으면 **3-way merge**

Fast-forward vs. 3-way merge

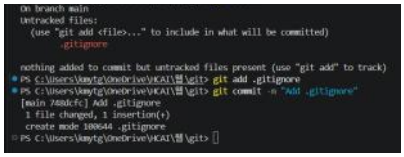


Merge conflict 해결

같은 파일의 같은 부분을 서로 다르게 수정하면 충돌이 발생

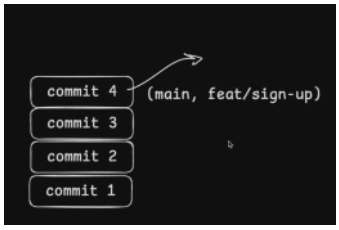
해결 방법
1. 충돌 파일 확인
2. 충돌 코드 직접 수정
3. 새로운 커밋 생성

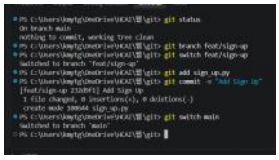
Git은 자동 병합을 시도하고, 판단이 필요한 경우만 충돌로 알려줌

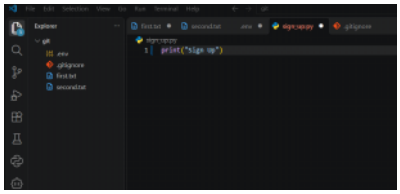


- Ignore 한게 어떻게 한건지.. id/pw있던 문서 어디감..





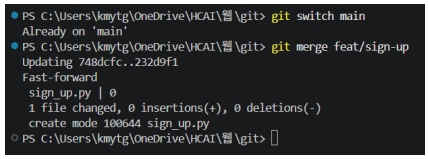


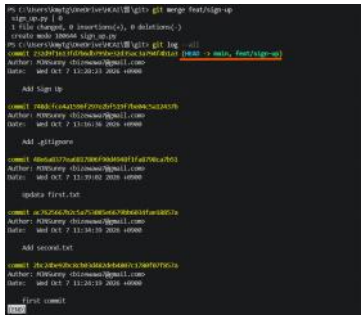






<-모든 커밋

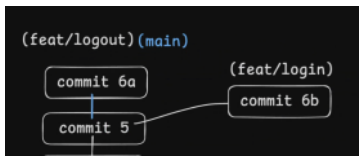








Fast-forward, 5번의 변경사항만 반영됐다.



6b merge 시 6a가 없어짐.

보는 커맨드!



Merge conflict 해결 실습

```
# Git 레포지토리 초기화
$ git init

$ echo "hello" > test.txt
$ git add .
$ git commit -m "first commit"
```

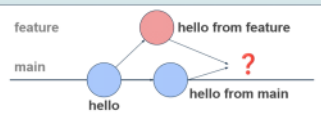

Merge conflict 해결 실습

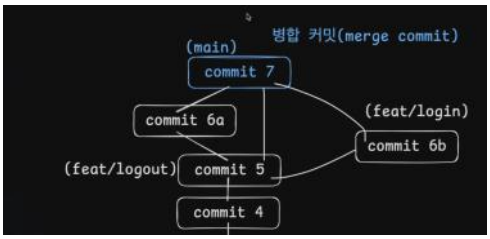
```
# feature 브랜치 생성
$ git branch feature
$ git switch feature

# 같은 파일 수정
$ echo "hello from feature" > test.txt
$ git add .
$ git commit -m "update from feature"
```

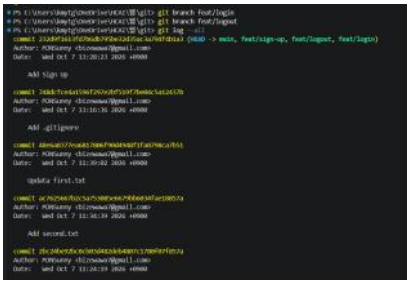

Merge conflict 해결 실습

```
# main에서 다시 수정
$ git switch main
$ echo "hello from main" > test.txt
$ git commit -am "update from main"
```





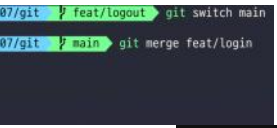
3 way merge (해결책)

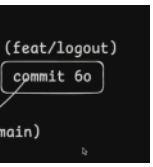




git log --all --graph

참고: <https://mermaid.live/edit#pako:eNpVjMEKwjAQRH9l2XMLtdZqcxPBk6AXb7mkyaYNJpuSbEBK6b-bVhH0NMybeTMrMFoFBoMcXbQ2eHIUBgbOTnaCwWiXMDjZ0Y1uMDjZ0Y1uMDjZ0Y1u



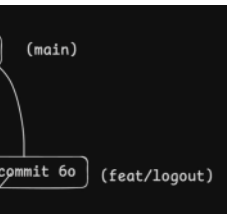


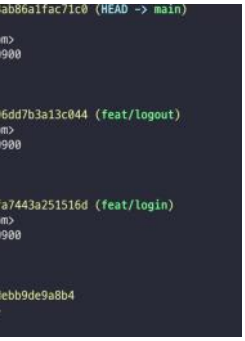


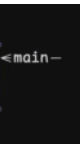


:wq

=> 3 way merge

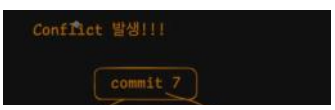


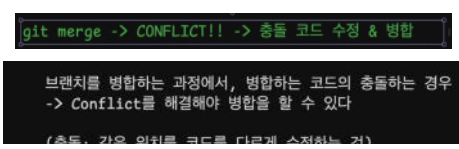












로컬과 원격 저장소를 동기화하는 명령어
- **git push:** 로컬 커밋을 원격 저장소로 올림
- **git pull:** 원격 변경 사항을 내려받아 로컬에 반영

1. 로컬 커밋을 원격 저장소로 업로드
   $ git push -u origin main

2. **GitHub** 웹에서 파일 추가 **&** 커밋 생성

3. 원격 저장소의 변경 사항 다운로드
   $ git pull



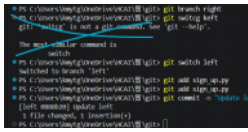
Clone vs. Fork

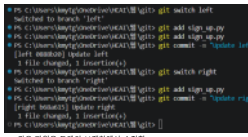
Clone
- 원격 저장소를 그대로 내 로컬로 복사
- 같은 저장소에 **push** 권한이 있을 때 사용
- 팀 내부 협업에 주로 사용

Fork
- 다른 사람의 저장소를 내 **GitHub** 계정으로 복사
- 원본 저장소에 직접 **push** 권한이 없을 때 사용
- 오픈소스 기여 방식

Main branch 만 남기고 삭제:







<-- 같은 파일을 두개의 브랜치에서 수정함

Pull Request

변경 사항을 원본 저장소에 반영해달라고 요청하는 것
- 변경 내용 설명
- 리뷰 요청
- 병합 전 검증 단계

git merge left / right:

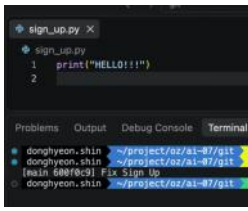


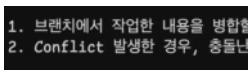
Code Review 기본

코드를 병합하기 전에 함께 확인하는 과정
- 버그 예방
- 코드 품질 유지
- 팀 규칙 공유

기본 원칙
- 코드 중심으로 리뷰
- 이유를 남긴 코멘트
- 승인 후 병합

때로는 완전히 새로운 코드를 쓸때도





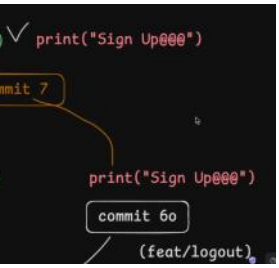
Issue 사용법

할 일, 버그 리포트, 기능 요청 등을 관리하는 기능
→ **PR**과 **Issue**를 연결하여 작업 관리

예시
- **Resolve: #1234**
- https://github.com/sqlalchemy/alembic/pull/1754

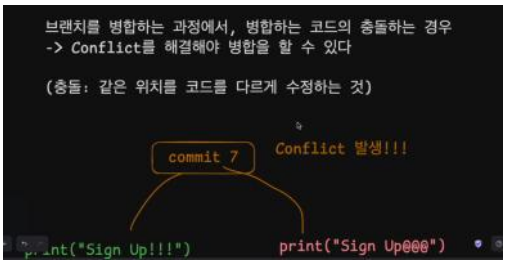


Left, right을 변경하는건 아님.
사실 git은 잘 삭제를 해줘야 하는 편.



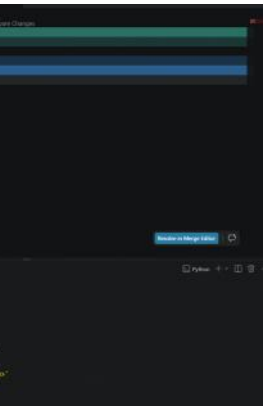


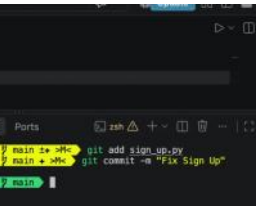
Git이 선택을 못함. 그래서 사람이 해야 함.



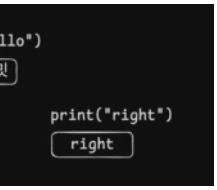








때는 git merge
코드를 수정하고 commit



협업 실습(Fork & Clone)

1. Issue 생성
2. 내 GitHub 계정으로 Fork
3. Clone으로 코드 가져오기
   a. git clone 내_포크_저장소_URL
4. 원본 저장소 연결
   a. git remote add upstream 원본_저장소_URL
5. 최신 코드 받기
   a. git pull upstream main
6. 브랜치 생성
   a. git switch -c feature/add-my-name

협업 실습(Collaborator)

1. Repository collaborator 등록
2. Branch protection rules 추가
   a. Require a pull request before merging
3. Clone으로 코드 가져오기
   a. git clone 원본_저장소_URL
4. 새로운 브랜치 생성 & 커밋
   a. git switch -c feature/mybranch & git commit -m “message”
5. 원격 브랜치에 push
   a. git push origin feature/mybranch
6. Pull Request 생성
   a. GitHub에서 feature/mybranch → main PR 생성

git show

특정 커밋 하나를 집중해서 살펴볼 때 사용하는 명령어

- 해당 커밋의 변경 내용(diff)
- 커밋 메시지, 작성자, 날짜 출력
- 코드 리뷰나 이력 확인 시

git show	(git show HEAD와 동일)
git show HEAD	현재 브랜치의 최신 커밋 내용 확인
git show 커밋해시	특정 커밋 하나를 지정해서 확인

git diff

두 상태 간의 차이점만 비교해서 보여주는 명령어

- 커밋 전에 변경 사항 점검
- 병합·충돌 원인 분석

git diff	Working Directory ↔ Staging Area
git diff --staged	Staging Area ↔ HEAD
git diff HEAD~1 HEAD	커밋 ↔ 커밋

git blame

파일의 각 줄이 누가, 언제, 어떤 커밋에서 수정됐는지 추적

- 버그 원인 추적 시
- “이 줄은 왜 이렇게 되어 있지?”

git blame main.py	main.py의 각 줄을 누가, 언제, 어떤 커밋에서 수정했는지 확인

git stash

현재 작업 중인 변경 사항을 임시로 치워두는 명령어

- 커밋하지 않고 작업을 잠시 중단
- 브랜치 이동이나 긴급 수정 시 유용

git stash	현재 작업 중인 변경 사항을 임시로 저장
git stash list	지금까지 저장된 stash 목록을 확인
git stash pop	가장 최근에 저장한 stash를 다시 적용하고 목록에서 제거
git stash apply	특정 stash를 제거하지 않고 적용만 하려면

git reset

브랜치의 기준을 과거 커밋으로 이동시키는 명령어(히스토리를 바꿈)

- **--soft:** 커밋만 취소 (변경 내용 유지)
- **--mixed(기본):** 커밋 + 스테이징 취소
- **--hard:** 전부 삭제 (주의)
- 협업 브랜치에서 사용시 주의

git reset --soft HEAD~1	HEAD만 한 커밋 이전으로 이동(변경 내용은 유지)
git reset --hard HEAD~1	HEAD 이동 + 스테이징 + 파일까지 모두 되돌림

git revert

특정 커밋을 되돌리는 새 커밋을 생성(히스토리를 보존)

- 기존 기록은 그대로 유지
- "이 커밋을 취소했다"는 기록이 남음
- 협업 중 안전하게 롤백할 때

git revert 커밋해시

git commit --amend

마지막 커밋을 고치는 옵션

- 가장 최근 커밋을 새 커밋으로 교체
- 커밋 메시지 수정 가능
- 변경 파일 추가/제거 가능

git commit --amend	마지막 커밋을 고친다
git commit --amend --no-edit	기존 커밋 메시지는 그대로 유지